

QRmanía

Plataforma de videojuegos de fiesta

Documento de Diseño de Juego (GDD)

Integrante: Franco Ezequiel Hereñú

Repositorio: github.com/francoheredev/Mafia

Contenido

- 1. Plataforma / Hub — GDD general del sitio
- 2. La Mafia — GDD del primer juego
- 3. Blind Shot — GDD del segundo juego

La Mafia — Plataforma (GDD)

GDD general del sitio/hub. Para el diseño de cada juego, ver *mafia.md* y *blind-shot.md*.

1. Concepto / Pitch

Un hub de **juegos de fiesta para jugar en persona, en red local**: una pantalla compartida (PC/TV) hace de tablero, y cada jugador usa su propio celular como control, sin instalar nada — se une escaneando un QR desde el navegador. La pantalla y los celulares quedan sincronizados en vivo por WebSockets. La plataforma no es un juego en sí: es el "esqueleto" (lobby, reconexión, chat, hub) sobre el que se montan los juegos concretos.

2. Arquitectura

El proyecto separa una capa de **plataforma genérica** (no sabe nada de ningún juego puntual) de los **juegos**, montados como plugins sobre ella:

```
la-mafia-lobby/
|-- server.js # bootstrap: Express, estáticos, require('./games'), listen
|-- platform/
| |-- core/ # rooms.js, registry.js, connection.js, timer.js — genérico
| |-- routes/ # hub.js (GET /, GET /api/games), game-pages.js
| |-- games/
| | |-- index.js # único archivo que crece: un require() por juego nuevo
| | |-- mafia/ # plugin.js, logic.js, roles.js, rules.js
| | |-- blind-shot/ # plugin.js, logic.js
| |-- public/
| | |-- hub/ # selector de juego
| | |-- platform/ # window.Platform: connect/roster/chat/history/rules
| | |-- games/<juego>;/ # screen.html/js, player.html/js, <juego>.css
```

Cada juego se registra con `registerGame({ id, createGameState, socketHandlers, onReconnect, remapPlayerId, onKick, chatChannels, staticRoutes, publicDir })` (ver `platform/core/registry.js`). Sumar un juego nuevo es: una carpeta en `games/<id>/`, una carpeta en `public/games/<id>/`, y una línea `require("./games/<id>/plugin")` en `games/index.js` — `platform/` no necesita tocarse.

3. Flujo de usuario

- 1 El anfitrión abre el **hub** (/) en la PC/TV y elige un juego.
- 2 Abre la **pantalla** de ese juego (/<juego>/screen), que crea una sala y muestra un QR.
- 3 Cada jugador escanea el QR con su celular (o entra a /<juego>/player a mano) y se une a la sala con su nombre e ícono.
- 4 Cuando hay suficientes jugadores conectados, la pantalla arranca la partida.
- 5 Si un jugador se desconecta a mitad de partida, no se lo borra de la sala: queda marcado como desconectado y puede reincorporarse (`player:rejoin`) — "reconexión congelada".

4. Stack tecnológico

- **Backend:** Node.js + Express 4 (routing/estáticos) + Socket.IO 4 (tiempo real). Sin base de datos: todo el estado vive en memoria por sala (`platform/core/rooms.js`).
- **Frontend:** JavaScript vanilla, sin framework y sin build step — HTML/ CSS/JS servidos como archivos estáticos.
- **Cliente:** la única librería de terceros visible es QRCode (para los QR de unión); todo lo demás (audio, partículas, canvas) es hecho a mano con APIs nativas del navegador (Web Audio API, Canvas 2D, Pointer Events, `navigator.vibrate`).

- **Testing:** scripts propios en Node (`tests/`, orquestados por `run-tests.js`) que levantan el servidor real y simulan clientes de Socket.IO — sin framework de test.
- Todo el código, comentarios y la interfaz están en **español**.

5. Catálogo de juegos

La Mafia	Deducción social (tipo Mafia/Hombres Lobo), Noche/Día por turnos	mafia.md
Blind Shot	Battle royale simultáneo y "a ciegas", con zona que se achica	blind-shot.md

6. Convenciones compartidas

- `window.Platform` (`public/platform/`): toolkit de cliente compartido por todos los juegos — conexión, roster de jugadores, chat, historial de eventos de la sala, y el modal de reglas.
- **Audio sintetizado:** ningún juego usa archivos de sonido — todo se genera en vivo con Web Audio API (ver, por ejemplo, el disparo de Blind Shot o los stingers narrativos de Mafia).
- **Paleta visual:** tema oscuro compartido vía variables CSS de `platform.css` (fondo oscuro, acento dorado, rojo para peligro/alertas), con iconografía basada en emoji en vez de arte/sprites propios.
- **Identificación de sala:** cada jugador se identifica por su `socket.id`, que cambia al reconectar — por eso todo juego implementa el hook `remapPlayerId` para "mudar" ese id en su propio estado.

7. Roadmap

- Sumar más juegos al hub (cada uno con su propio GDD en esta misma carpeta).
- El hub hoy solo muestra el id del juego "prettificado" (`prettifyId` en `public/hub/hub.js`) — no hay descripción ni miniatura por juego; se podría sumar un título/ícono/descripción propios por juego al registry para una tarjeta más rica en el hub.

La Mafia (GDD)

*Numeración de secciones alineada a las referencias que ya existen en el código (roles.js, logic.js, plugin.js, README) — por ejemplo "sección 6 del GDD" o "Fase 1 del GDD" apuntan a las secciones de acá. Ver también el **GDD de la plataforma**.*

1. Concepto / Pitch

Juego de deducción social tipo Mafia/Hombres Lobo, jugado en persona: una pantalla compartida hace de narrador/tablero y cada jugador tiene un rol secreto que gestiona desde su celular. De noche, la Mafia elige en secreto a quién eliminar mientras el resto de roles con poder actúa en privado; de día, todo el pueblo discute en voz alta, acusa y juzga. Nadie sabe con certeza quién es quién — se descubre (o no) discutiendo.

2. Objetivo y condición de victoria

Chequeo binario Mafia/Ciudad, evaluado después de cada muerte (de noche o por ejecución):

- **Gana la Ciudad** si ya no queda ningún mafioso vivo.
- **Gana la Mafia** si su cantidad supera **estrictamente** a la del resto (los Independientes, como el Bufón, cuentan como "buenos" para este chequeo).
- **Empate** si ambos bandos llegan a cero jugadores a la vez (posible por la venganza en cadena del Cazador).
- Al terminar, la pantalla revela el rol de todos (vivos y muertos).

El **Bufón** además tiene su propia forma de ganar, independiente del chequeo binario: **gana solo si el pueblo lo vota y lo ejecuta durante el Juicio** (resolveTrial en logic.js) — no importa qué bando iba ganando en ese momento.

3. Ambientación y narrativa

3.1 Ambientación

Reskin narrativo de aldea de fantasía: la Mafia se presenta como una manada de **Hombres Lobo** (Padrino -> "Lobo Alfa", Mafioso -> "Hombre Lobo"), y el resto de los roles tiene su propio nombre narrativo además del técnico (ver tabla de roles, sección 6.1). La pantalla narra cada resolución de Noche/Día en varios pasos con pausas — niebla, la mafia acechando, el nombre de la víctima si hubo muerte — en vez de un resultado plano, antes de dejar fija la pantalla de resultado.

Pendiente: pulir animaciones/arte específicos de esta ambientación (ver sección 8, Roadmap).

4. Flujo de partida

Ciclo Noche -> Día que se repite hasta que se declara un bando ganador:

- **Fase 0 — Asignación de roles:** al tocar "Empezar partida" en la pantalla (habilitado con 6-10 jugadores conectados), el servidor sortea el catálogo de roles (sección 6) y le manda a cada celular su rol en privado. Los mafiosos ven en su tarjeta a sus cómplices.
- **Fase 1 — Noche:** la pantalla entra en modo "cae la noche" con cronómetro (60s). La Mafia elige víctima con **líder rotativo** (rota entre los mafiosos vivos noche a noche — el resto del equipo ve quién decide y puede sugerir, pero no es vinculante); el Detective/Vidente investiga en privado a un jugador (el Padrino se ve como inocente); el Médico protege a alguien. Si la Mafia tiene Bruja o Carnicero, actúan además con su poder propio (ver sección 6.1). Si matan al Cazador, dispara automáticamente contra otro jugador vivo al azar. Una vez que actuaron todos los que tenían algo para hacer, la noche se acorta a un margen corto en vez de esperar el timeout completo.

- **Fase 2 — Amanecer:** se resuelve la noche (muerte o "nadie murió"/ "el Médico llegó a tiempo") y se chequea la condición de victoria antes de seguir.
- **Fase 3 — Discusión:** el pueblo debate en voz alta mirando la pantalla compartida, con un cronómetro (60s por defecto) que la pantalla puede cortar antes con un botón.
- **Fase 4 — Votación:** cada celular vota en privado a quién acusar, o se abstiene (30s). Si hay empate entre los más votados, **se sortea entre los empatados** — el sorteado sigue el mismo camino que cualquier acusado (no hay eliminación directa por empate). Solo si nadie recibió ningún voto, no hay acusación ese día.
- **Fase 5 — Defensa:** el acusado (por votos o por sorteo) tiene la palabra frente a toda la sala; también cortable desde la pantalla (30s por defecto).
- **Fase 6 — Juicio:** el resto vota culpable/inocente en privado (30s); el acusado no vota su propio juicio. Si "culpable" supera a "inocente", se ejecuta. **Nadie revela su rol al morir** (ni de noche ni por ejecución) — el sistema nunca lo anuncia, se descubre discutiendo. Si no hay ganador todavía, vuelve a caer la noche (Fase 1).

La pantalla le avisa al servidor (day : advance) apenas termina de mostrar su narrativa, así nunca se pisa con la fase siguiente — hay timers de respaldo (NIGHT_RESULT_MS/DAY_RESULT_MS) que solo entran si la pantalla nunca avisa.

5. Stack tecnológico

Todo en JavaScript: Node + Socket.IO en el servidor, HTML/CSS/JS plano en el cliente — sin motor de juego (Godot u otro) ni build step. Ver el detalle completo del stack, compartido con toda la plataforma, en el **GDD de la plataforma, sección 4**.

6. Roles

6.1 Catálogo de roles (MVP)

10 roles, cada uno con nombre técnico, nombre narrativo (sección 3.1) e ícono. Los mafiosos se conocen entre sí desde la Fase 0.

Padrino	Lobo Alfa	Mafia	Líder de la mafia; el Detective lo ve como inocente si lo investiga.
Mafioso	Hombre Lobo	Mafia	Participa en la elección nocturna de la víctima.
Bruja	Bruja	Mafia	Además del voto colectivo, de noche puede revelar a toda la Mafia el rol exacto de un jugador.
Carnicero	Carnicero	Mafia	Además del voto colectivo, de noche puede silenciar a un jugador (no puede votar en la acusación del Día siguiente).
Detective	Vidente	Ciudad	De noche, investiga en privado a un jugador para saber si es de la Mafia.
Médico	Curandero/a	Ciudad	De noche, protege a alguien de un ataque.
Cazador	Cazador	Ciudad	Sin acción nocturna propia; si lo matan, dispara automáticamente a otro jugador vivo al azar.

Aldeano	Aldeano	Ciudad	Sin habilidad especial — solo su voto y su palabra.
Intendente	Intendente	Ciudad	Sin habilidad nocturna, pero su voto vale doble (acusación y juicio).
Lycan	Lycan	Ciudad*	Sin habilidad nocturna; la primera vez que intentan matarlo, sobrevive y pasa a jugar para la Mafia.
Bufón	Bufón	Independiente	Gana solo si el pueblo lo vota y lo ejecuta durante el Juicio (sección 2).
Amante ×2	Amante	Ciudad	Sin habilidad especial, pero conoce en secreto la identidad de su Amante (el otro jugador con este rol).

* El Lycan cuenta como Ciudad hasta que sobrevive a un intento de asesinato — desde ese momento pasa a jugar para la Mafia (se suma a `mafiaOrder` y a la lista de cómplices del resto de la Mafia).

Ajuste sobre el catálogo original: el Aldeano se agregó porque el MVP original solo tenía 3 roles Ciudad con habilidad (Detective, Médico, Cazador) y hacía falta completar el bando Ciudad en partidas de 7+ jugadores.

6.2 Escalado por cantidad de jugadores

Ajuste a la tabla original: la tabla no reservaba un lugar fijo para el Bufón como Independiente — quedó corregida acá para que cada tramo reparta el mismo total de Mafia/Ciudad/Independiente que la tabla original tenía a esa cantidad de jugadores, "reskineando" solo roles de relleno (aldeanos extra, el 2º mafioso) en roles nuevos a medida que crece el grupo, sin inflar el poder de ningún bando.

6	2	3	1	Padrino, Mafioso	Detective, Médico, Cazador
7	2	4	1	Padrino, Mafioso	Detective, Médico, Cazador, Intendente
8	2	5	1	Padrino, Mafioso	Detective, Médico, Cazador, Intendente, Lycan
9	3	5	1	Padrino, Mafioso, Bruja	Detective, Médico, Cazador, Intendente, Lycan
10	3	6	1	Padrino, Bruja, Carnicero	Detective, Médico, Cazador, Intendente, Lycan, Aldeano

A partir de 8 jugadores, cada partida tiene además un 50% de chance (`LOVERS_CHANCE`) de reemplazar los lugares de Intendente y Lycan por 2 Amantes (mismo total de Ciudad, sin tocar el resto de la tabla) — así esos dos roles no desaparecen del juego para siempre en las salas de 8+.

7. UI/UX

- **Ver tu rol en cualquier momento:** botón fijo () en el celular que abre la tarjeta de rol en una ventana superpuesta, incluso después de morir.
- **Chat:** la sala tiene un canal general, más un canal privado solo para los mafiosos vivos.

- **Reconexión:** si un jugador se desconecta, queda "congelado" (sigue a `live`, marcado `connected: false`) y puede reincorporarse; al reconectar recibe de nuevo su rol y el estado exacto de la fase en la que está la sala (`mafiaOnReconnect` en `logic.js`).
- **Tutorial in-app:** `games/mafia/rules.js` (`GENERAL_RULES`) contiene el contenido del modal de reglas (objetivo, noche, discusión, votación/ juicio, el secreto de la muerte), disponible desde el celular y la pantalla en cualquier momento.

8. Roadmap / Qué falta

- Pulir animaciones/arte con la ambientación de aldea de fantasía (sección 3.1) — es intencionalmente lo último de la lista, después de tener el ciclo completo de juego funcionando.
- Hacer configurable por sala si se revela o no el rol al morir (hoy nunca se revela, por diseño — sección 4, Fase 6).

Blind Shot (GDD)

*Segundo juego de la plataforma — sin GDD previo, así que la numeración de este documento es libre (a diferencia de **mafia.md**, que respeta secciones ya citadas en el código). Ver también el **GDD de la plataforma**.*

1. Concepto / Pitch

Battle royale simultáneo y "**a ciegas**": en cada ronda, todos los jugadores vivos mueven y apuntan en privado desde su celular, sin ver a los demás — recién al cerrar la ronda se revelan juntas todas las posiciones y los disparos. Una zona rectangular que se va achicando ronda a ronda fuerza el enfrentamiento. Gana el último en pie.

2. Objetivo y condición de victoria

- **Gana** el único jugador que sigue vivo al resolverse una ronda.
- **Empate** si una cadena de eliminaciones deja a 0 jugadores vivos en la misma ronda (`checkWinner` en `logic.js`, mismo criterio que la venganza del Cazador en Mafia).
- Mínimo 2 y máximo 16 jugadores por partida (`MIN_PLAYERS/MAX_PLAYERS`).

3. Loop de ronda

- 1 **Movimiento a ciegas** (20s reales; 800ms en tests): cada celular es un <canvas> con Pointer Events como control dual-touch — el primer dedo es un joystick flotante para moverse dentro de la zona vigente, el segundo dedo apunta libremente (se ve un láser con el ángulo actual). Soltar el dedo de puntería no borra el ángulo: queda "vivo" para la próxima ronda. Nadie ve al resto de los jugadores durante esta fase — ni siquiera la pantalla dibuja sus posiciones.
- 2 **Disparo obligatorio**: todo jugador vivo dispara en cada ronda, tenga o no una puntería explícita esta vez. Si nunca tocó el segundo dedo, el ángulo se resuelve con una cadena de respaldo (`resolveAim`): (a) ángulo explícito si mandó uno, (b) dirección de movimiento si se movió más de un umbral mínimo, (c) el último ángulo conocido (0 en su primera ronda).
- 3 **Resolución** (`resolveRoundWithOrder`): se sortea un orden aleatorio entre los jugadores vivos al empezar la ronda. En ese orden, cada disparo se resuelve como un rayo con un **corredor de ancho fijo** (40 unidades a cada lado de la línea de puntería, no una tolerancia angular — una tolerancia angular sería trivial a rango largo e imposible de cerca): entre los objetivos dentro del corredor, impacta al más cercano en la línea de tiro. Si a un jugador lo matan antes de que le toque disparar en este orden, su disparo no sale (muertes encadenadas posibles en la misma ronda).
- 4 **Revelación** (8s reales; 300ms en tests): se muestran primero todas las posiciones finales a la vez, y después los disparos se reproducen en secuencia.
- 5 **Achique de zona**: si no hay ganador todavía, la zona se achica un factor fijo (0.85×) en ambos ejes por igual (con un piso mínimo por eje que respeta la misma proporción, así nunca se deforma), y cualquier sobreviviente que haya quedado afuera del nuevo rectángulo es empujado hacia adentro. Vuelve a la fase 1.

La pantalla puede cortar antes de tiempo la espera de la revelación (`round:advance`) y pasar ya a la próxima ronda; a diferencia de Mafia, el movimiento nunca se corta antes de tiempo aunque todos ya hayan mandado su jugada — el diseño es que dura siempre los 20s completos.

4. La zona

Es un **rectángulo**, no un círculo — decisión tomada después de probarlo, a pedido explícito del usuario, para que ocupe toda la pantalla del celular en vertical. Mantiene siempre la proporción 9:16 (`ZONE_ASPECT`) de una pantalla de celular, tanto en el tamaño inicial del arena como en cada achique y en su piso mínimo.

5. Controles / input

- **Primer dedo (mover):** joystick flotante que aparece donde se apoya el dedo, limitado a la zona vigente.
- **Segundo dedo (apuntar):** ángulo libre en 360°, mostrado con un láser rojo (#ff3b3b) siempre visible mientras se sostiene.
- El servidor nunca confía en el clamp del cliente: siempre re-clampea la posición recibida a la zona vigente antes de guardarla (`round:submit` en `plugin.js`).

6. Presentación en pantalla

- **Marcadores:** cada jugador se dibuja con el ícono que eligió al unirse (no un punto genérico), tanto en el mini-mapa del celular como en la arena de la pantalla compartida.
- **Coreografía de la revelación:** primero se muestran todas las posiciones finales a la vez (pausa de 1.6s — "así quedaron parados", a pedido del usuario), y recién después los disparos se reproducen uno por uno (cadencia de 1.1s) con un flash en el ícono del tirador, la línea del láser hasta el punto exacto de impacto, y los objetivos eliminados pasando a 30% de opacidad.
- **Partículas** (sistema propio, sin librería): ráfaga de fogonazo (cono angosto en torno al ángulo de puntería, tonos naranja/amarillo) en el tirador, y ráfaga de impacto (omnidireccional, roja) en el punto de impacto — con fricción/desvanecimiento frame a frame.
- **Sonido de disparo sintetizado** (Web Audio, sin archivos de audio): un "crack" de ruido filtrado (pasabanda ~1800Hz) junto con un "thump" de onda senoidal descendente (140Hz -> 40Hz) — misma filosofía de síntesis que los stingers narrativos de Mafia.
- **Zona:** se dibuja el contorno completo del arena (tenue) y el rectángulo de la zona vigente resaltado en dorado, achicándose ronda a ronda.

7. UI/UX

- **Chat:** un único canal "general", abierto a todos los conectados en todo momento — a diferencia de Mafia, acá no hay bandos secretos: un jugador eliminado pasa a ser espectador de la misma conversación.
- **Espectadores:** al reconectar, un jugador ya eliminado recibe `round:spectator` en vez de estado de juego.
- **Tutorial in-app:** modal de reglas con 5 secciones cortas (movimiento a ciegas, joystick, puntería, disparo obligatorio, zona que se achica).

8. Stack técnico

Mismo stack que el resto de la plataforma (ver **GDD de la plataforma, sección 4**): Node + Socket.IO en el servidor, canvas + Pointer Events en el cliente, sin librerías extra más allá del QRCode heredado del hub. Toda la simulación de ronda (`resolveRoundWithOrder`) es pura (no toca io ni sockets), lo que permite testearla con fixtures armados a mano en `tests/blind-shot/test-round-resolve.js`.

9. Roadmap

No hay todavía una lista de pendientes documentada para este juego (a diferencia de Mafia). Completar esta sección a medida que surjan próximos pasos.